

ACA LAB Project Report

Team:

Peta Shanmuga Teja 25CS06004

Khem Singh 25CS06004

Implementation and Analysis of a Victim Cache in SimpleScalar

1. Objective

The objective of this project is to integrate a **victim cache** between the Level-1 (L1) and Level-2 (L2) data caches in the SimpleScalar simulator (`sim-cache.c`) and to analyze its impact on memory performance compared to the baseline configuration without a victim cache.

2. Introduction

A victim cache is a small, fully associative cache placed between the L1 and L2 caches. It temporarily stores recently evicted blocks from the L1 cache, reducing conflict misses. When the processor experiences an L1 miss, the victim cache is checked before accessing L2. If the data is found there (a *victim hit*), access latency is significantly reduced.

3. Implementation Details

All modifications were made in the `sim-cache.c` file of *SimpleScalar-3.0*:

- Declared a new cache pointer:
`static struct cache_t *victim_cache = NULL;`
- Added a configuration variable for victim cache entries:
`static int victim_entries;`
- Registered a new command-line option:
`-victim:entries`
This allows users to enable or disable the victim cache and set its size dynamically.
- Updated the `dl1_access_fn` function to check the **victim cache** on L1 misses before going to L2.

- Created a new victim cache instance between L1 and L2:
It is **fully associative**, with **one set** and **associativity equal to the number of victim entries**.
The hit latency was set to 1 cycle.
- Renamed the cache to "**victim_%d**" instead of "**victim:%d**" to ensure correct parsing of derived statistics.
- Adjusted the hit latencies of **dl2** and **il2** caches from 1 to 6 cycles to simulate realistic multi-level cache behavior.
- Registered victim cache statistics for reporting:
`cache_reg_stats(victim_cache, sdb);`

4. Command format:

```
./sim-cache -victim:entries 0 -cache:dl1 dl1:128:32:1:l -cache:dl2 ul2:1024:64:4:l
../Labs/Pisa/anagram
./sim-cache -victim:entries 4 -cache:dl1 dl1:128:32:1:l -cache:dl2 ul2:1024:64:4:l
../benchmarks/go.alpha 50 9 <../benchmarks/2stone9.in>
```

5. effective missrate calculation

```
if victim entries = 0 :
    effective miss rate = dl1 miss rate
if victim entries > 0 :
    victim cache misses/dl1 cache access
```

6. Observations and Results

	victim_entries	L1_Miss_Rate	Victim_Miss_Rate	Effective Miss Rate
anagram	0	0.1098	0	0.1098
	2	0.1098	0.3375	0.063
	4	0.1098	0.3325	0.0621
test-math	0	0.0211	0	0.0211
	2	0.0211	0.4254	0.0145
	4	0.0211	0.4147	0.0141
test-fmath	0	0.0387	0	0.0387
	2	0.0387	0.3393	0.0218
	4	0.0387	0.314	0.0202
go.alpha	0	0.1373	0	0.1373
	2	0.1373	0.6876	0.1286
	4	0.1373	0.6506	0.1217

Effective Miss Rate vs. Victim Entries by Category

